

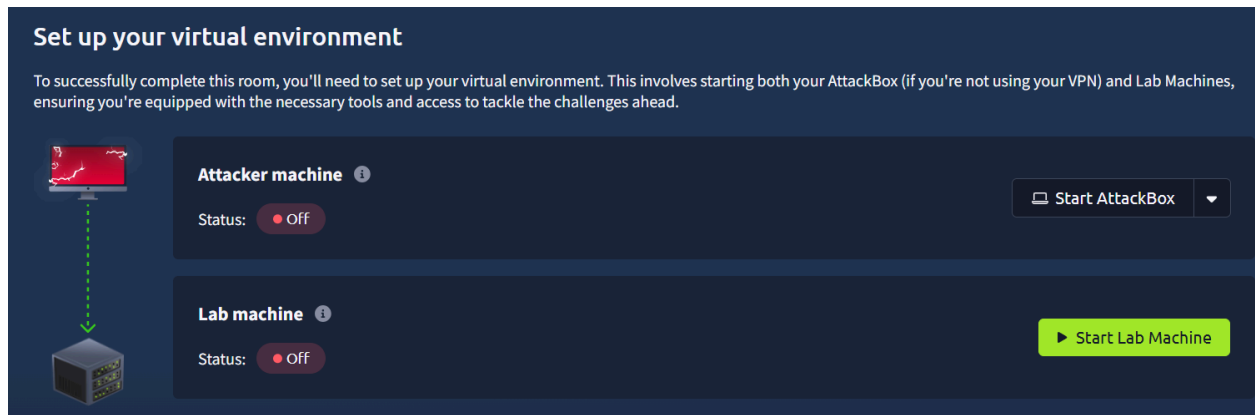
Challenge Setup and Initial Connection:

At the beginning of the challenge, we are provided with the **source code**. We can download the source code file using the download option provided in the challenge.

The challenge also provides the following information:

The server is listening on **TCP port 1337** and can be connected using **Netcat (nc)** or any other tool

Once we start the **attacker machine** and the **lab machine**, we can establish a connection to the lab machine using Netcat.



Set up your virtual environment

To successfully complete this room, you'll need to set up your virtual environment. This involves starting both your AttackBox (if you're not using your VPN) and Lab Machines, ensuring you're equipped with the necessary tools and access to tackle the challenges ahead.

Attacker machine ⓘ

Status: ● Off

Start AttackBox ▼

Lab machine ⓘ

Status: ● Off

Start Lab Machine

Connecting to the Lab Machine

Use the following command:

```
nc <IP-address-lab> <port-number>
```

For example:

```
nc 10.10.10.10 1337
```

Here:

- **nc** — Netcat, a command-line networking utility.
- **<IP-address>** — IP address of the lab machine.
- **<port-number>** — Port on which the server is listening, in this case **1337**.

Viewing Netcat Options

To learn more about Netcat and its available options, use the **man** (manual) command: **man nc**

Examining the source code

1. The program starts at the main function, at the start it binds to port 1337 on all interfaces and sets the RequestHandler class to handle all incoming requests.

```
1  if __name__ == '__main__':
2      socketserver.ThreadingTCPServer.allow_reuse_address = True
3      server = socketserver.ThreadingTCPServer(('0.0.0.0', 1337), RequestHandler)
4      server.serve_forever()
```

RequestHandler:

```
class RequestHandler(socketserver.BaseRequestHandler):
    def handle(self):
        start(self.request)
```

This handles a new client connection. When a client connects:

Client

↓

RequestHandler

↓

handle()

↓

start()

The start() function then runs the challenge.

2. Generate a random **5-character key**.

```
def start(server):
    res = ''.join(random.choices(string.ascii_letters + string.digits, k=5))
    key = str(res)
    hex_encoded = setup(server, key)
    send_message(server, "This XOR encoded text has flag 1: " + hex_encoded + "\n")

    send_message(server, "What is the encryption key? ")
    key_answer = server.recv(4096).decode().strip()
```

This controls the main challenge interaction. It:

- Random key is generated using:
`random.choices(string.ascii_letters + string.digits, k=5)`
*{string.ascii_letters:(abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ)
and string.digits (0123456789) as the character set.}*
- Calls `setup()` to encrypt the fake flag.
- Sends the encrypted text to the client and asks the client for the key.
- Compare the received key with the actual key.
- Send the flag2 if the key is correct.

3. Use the key to XOR-encrypt a flag.

```
def setup(server, key):
    flag = 'THM{thisisafakeflag}'
    xored = ""
    for i in range(0, len(flag)):
        xored += chr(ord(flag[i]) ^ ord(key[i%len(key)]))
    hex_encoded = xored.encode().hex()
    return hex_encoded
```

This is the main ****XOR encryption function****. It:

- Takes the flag and encryption key.
- XORs each flag character with the corresponding key character.

- Repeats the key using ``i % len(key)``.
- Converts the encrypted result to hexadecimal and returns it.

The main XOR operation is: `ord(flag[i]) ^ ord(key[i % len(key)])`

{`ord()` converts characters to numbers, `^` performs XOR, and `chr()` converts the result back to a character}

`i % len(key)` ensures the 5-character key repeats across the entire flag.

This is known as **repeating-key XOR**.

4. Now, back to the start function. First, it will print the XOR encrypted and hex encoded flag returned from the setup function.

```
send_message(server, "This XOR encoded text has flag 1: " + hex_encoded
+ "\n")
```

5. Ask for the encryption key .If the key is correct, it prints the second flag from `flag.txt`.

```
1  send_message(server, "What is the encryption key? ")
2  key_answer = server.recv(4096).decode().strip()
3
4  try:
5      if key_answer == key:
6          send_message(server, "Congrats! That is the correct key! Here is flag 2: " +
7              server.close()
8      else:
9          send_message(server, 'Close but no cigar' + "\n")
10         server.close()
11 except:
12     send_message(server, "Something went wrong. Please try again. :)\n")
13     server.close()
```

XOR Function

XOR (Exclusive OR) is a bitwise operation that compares two values.

The basic rule is:

0	0	0
0	1	1
1	0	1
1	1	0

One important property of XOR is that the same key can be used to decrypt the data:

Plaintext XOR Key = Ciphertext

Ciphertext XOR Key = Plaintext

For example:

5 XOR 3 = 6

6 XOR 3 = 5

Recovering the Key

The ciphertext we worked with was:

1b3d380d327e141918360a0d0137363b41161d210e1b07452323390c1e173d010c46373d0d3a043f

From the code we get to know that XOR key has a length of **5** and the flag has the format of **THM{...}**.

Recall the XOR property:

Plaintext XOR Key = Ciphertext

Ciphertext XOR Key = Plaintext

Knowing this, we can apply it to our current case to recover the first 4 characters of the key, since we know the first 4 characters of the flag ('THM{') and the encrypted flag.

The hexadecimal ciphertext is first converted back into bytes, and then each byte is XORed with the corresponding key character.

Using python :

```
from pwn import xor

encrypted_flag =
bytes.fromhex('1b3d380d327e141918360a0d0137363b41161d210e1b07452323390c1e173d010c46373d0d3a043f')

xor(encrypted_flag[:4], b"THM{")
```

For last character:

```
xor(encrypted_flag[-1], b"}")
```

How xor works:

T H M { = 54 48 4d 7b

Now applying XOR:

1b XOR 54 = 4f

*/*hex is converted into bytes “/*

3d XOR 48 = 75

38 XOR 4d = 75

0d XOR 7b = 76

4f 75 75 76 = O u u v

Notice the length of the flag is 40 .This means the last character of the flag will be XOR'ed with the fifth character of the key.We also know the last character of the flag ('}')

} = 7d

3f XOR 7d = 42 = B

Hence the key we identified was:

OuuuB

So once we know OuuvB, we can reverse the XOR operation and recover the plaintext flag.